

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 13

Traversing a Binary Tree

Today

- Traversal (pre/in/post-order) without recursion
 - Or a stack. With $O(1)$ additional memory.
- Traversal starting at any node
- A closer look at in-order, pre-order and post-order traversal
 - And again, traversal starting at any node

Traversing a Binary Tree

- This is recursive. Can we write it as a loop with $O(1)$ extra space?
- Not tail recursion (two calls)
- What is on the stack?
 - Below any node, is its parent node
 - But we can find out the parent node from the tree itself!
 - Also on the stack: the program counter (what is the next action)

```
void explore(node* tree) {  
    if(tree==nullptr) return;  
    explore(tree->left);  
    std::cout << tree->key << " ";  
    explore(tree->right);  
}
```

in-order

```
void explore(node* tree) {  
    if(tree==nullptr) return;  
    std::cout << tree->key << " ";  
    explore(tree->left);  
    explore(tree->right);  
}
```

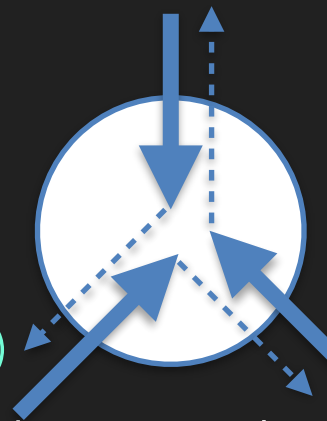
pre-order

```
void explore(node* tree) {  
    if(tree==nullptr) return;  
    explore(tree->left);  
    explore(tree->right);  
    std::cout << tree->key << " ";  
}
```

post-order

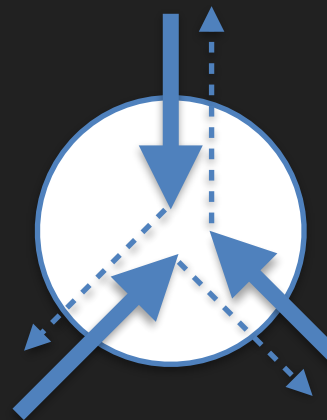
Traversing a Binary Tree: Non-Recursive

- When traversing the tree, we "enter" a node at 3 points:
 - Enter from the parent (and exit to the left child)
 - Re-enter when the left child returns (and exit to the right)
 - Re-enter when the right child returns (and exit to the parent)
- Current state of traversal (what gets stored in the stack) is fully determined by the current node, *and* which of the three ways we entered
- When re-entering a parent node, the point of entry (from left or from right) is determined by the current node
- So, when popping a state in the stack, we already know what is under it!
 - We don't need the stack! (But we do need parent pointers)



Traversing a Binary Tree: Non-Recursive

```
enum from_t { PARENT, LEFT, RIGHT };
struct state {
    node* p; from_t from; // p != nullptr except at the end
    void step() { // assumes p != nullptr
        switch(from) {
            case PARENT: // go left
                if(p->left) { p=p->left; from = PARENT; return; }
                from = LEFT; return; // visit+return from leaf
            case LEFT: // go right
                if(p->right) {p=p->right; from = PARENT; return; }
                from = RIGHT; return; // visit+return from leaf
            case RIGHT: // return to parent
                from = (p->parent && p == p->parent->left) ?
                    LEFT:RIGHT; // p->parent can be nullptr
                p = p->parent;
        }
    }
};
```

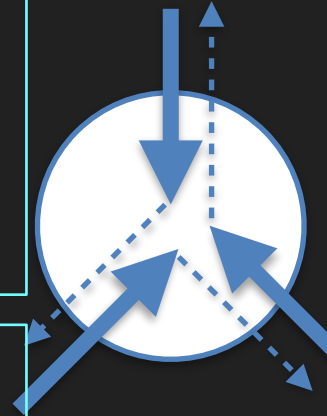


Traversing a Binary Tree: Non-Recursive

```
void explore_inorder(node* root) {  
    for(state st={root,PARENT}; st.p; st.step()) {  
        if(st.from == LEFT) std::cout << st.p->key << " ";  
    }  
}
```

```
void explore_preorder(node* root) {  
    for(state st={root,PARENT}; st.p; st.step()) {  
        if(st.from == PARENT) std::cout << st.p->key << " ";  
    }  
}
```

```
void explore_postorder(node* root) {  
    for(state st={root,PARENT}; st.p; st.step()) {  
        if(st.from == RIGHT) std::cout << st.p->key << " ";  
    }  
}
```



Attendance Break!

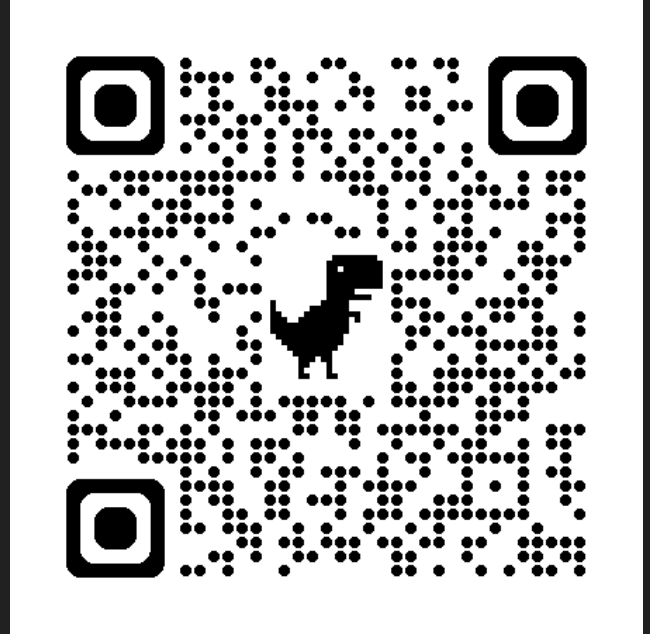
Login using CSE LDAP id at
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

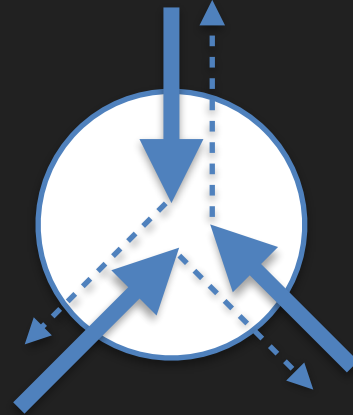
**If you are not in the classroom, a report will
be sent to DDAC!**



Traversing a Binary Tree: Next

- Finding the next node in the traversal sequence (in the order printed)

```
node* next_inorder(node* p) {  
    if(!p) return nullptr;  
    state st{p, LEFT}; // pretend p just printed  
    do st.step(); while(st.p && st.from != LEFT);  
    return st.p;  
}
```



- Traversing starting at a given node

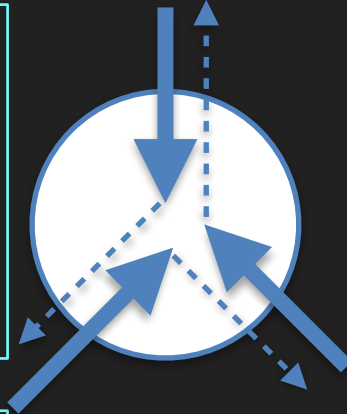
```
void explore_in(node* begin, node* end=nullptr) { // excluding end  
    for(node* curr=begin; curr!=end; curr=next_inorder(curr))  
        std::cout << curr->key << " ";  
}
```


Traversing a Binary Tree: Next

- Finding the next node in the traversal sequence (in the order printed)

```
node* next_preorder(node* p) {  
    if(!p) return nullptr;  
    state st{p, PARENT}; // pretend p just printed  
    do st.step(); while(st.p && st.from != PARENT);  
    return st.p;  
}
```

```
node* next_postorder(node* p) {  
    if(!p) return nullptr;  
    state st{p, RIGHT}; // pretend p just printed  
    do st.step(); while(st.p && st.from != RIGHT);  
    return st.p;  
}
```



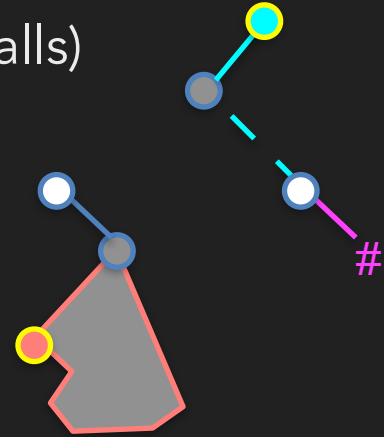
Traversing a Binary Tree: Next, Optimised

- A more direct version (avoiding multiple step function calls)

```
node* next_inorder(node* p) {  
    if(!p) return nullptr;  
    if(p->right) return first_inorder(p->right);  
    else return next_up(p);  
}
```

```
node* first_inorder(node* p) { // p not nullptr  
    while(p->left) {p = p->left;}  
    return p;  
}
```

```
node* next_up(node* p) { // p not nullptr  
    while(p->parent && p->parent->right==p) {p = p->parent;}  
    return p->parent;  
}
```

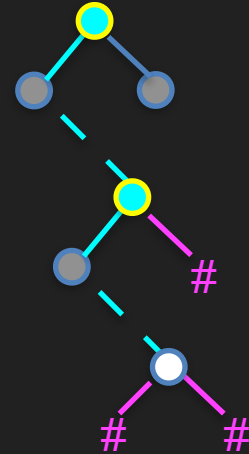


In a BST, this function computes the minimum in the subtree

Traversing a Binary Tree: Next, Optimised

- A more direct version (avoiding multiple step function calls)

```
node* next_preorder(node* p) {  
    if(!p) return nullptr;  
    if(p->left) return p->left;  
    if(p->right) return p->right;  
    do { p = next_up(p) } while (p && !p->right);  
    return p ? p->right: nullptr;  
}
```

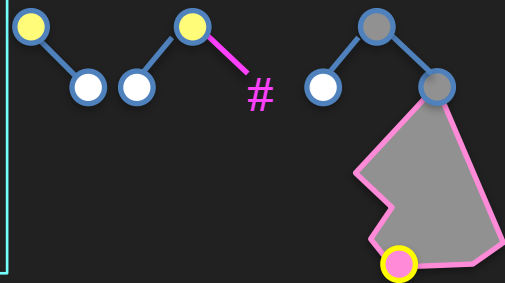


```
node* next_up(node* p) { // p not nullptr  
    while(p->parent && p->parent->right==p) {p = p->parent;}  
    return p->parent;  
}
```

Traversing a Binary Tree: Next, Optimised

- A more direct version (avoiding multiple step function calls)

```
node* next_postorder(node* p) {  
    if(!p || !p->parent) return nullptr;  
    if(p==p->parent->right || !p->parent->right)  
        return p->parent;  
    return first_postorder(p->parent->right);  
}
```



```
node* first_postorder(node* p) { // p not nullptr  
    while(p->left || p->right) {p = p->left ? p->left : p->right;}  
    return p;  
}
```

Summary

- Using parent pointers, can traverse a binary tree with $O(1)$ extra space
 - in-order, pre-order or post-order
- Can start from any node and continue traversal
 - `next()` function
- We saw two approaches to the next function
 - A clean, uniform approach using `state` and `step`
 - An optimised version handling different traversal orders differently